



League
Rescue Maze

Region
Japan

Team Info



Sponsors



About Team

Tuton is a team of three high school students from Japan competing in RoboCupJunior Rescue Maze.

Until the 2025 season, we participated in Rescue Line, where we developed strong skills in reliable hardware design and robust software engineering. Building on this experience, we have taken on the challenge of Rescue Maze this year.

Beyond achieving success in competition, we are committed to contributing to the community. We actively share our knowledge through technical articles and openly publish robot data and development results. For more details, please scan the QR codes displayed at the top of this poster.

This World Championship will be the final competition for our team. While striving for the championship title, we also hope to connect with competitors from around the world, exchange ideas, and gain valuable experiences.

Awards

RoboCupJunior Japan Open 2026 [Maze]

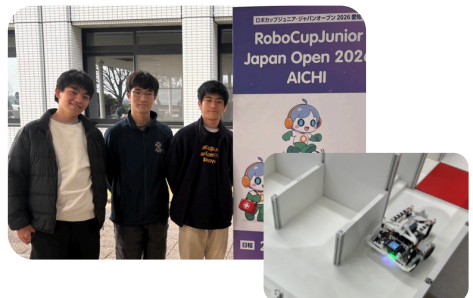
- 2nd Place
- Best Poster Award
- Outstanding Software Award

RoboCupJunior Japan Open 2025 [Line]

- 2nd Place
- Outstanding Hardware Award



Japan Open 2025



Japan Open 2026

Members



Shuji Igarashi

Captain, PCB, Software

I have been involved in RCJ since 2023. As Captain, I led project management and system integration. I also designed and manufactured our custom PCBs and contributed to the software, including the exploration algorithm.



Ryouta Koudu

Hardware

I have been in RCJ since 2023. As the Hardware Chief, I designed the robot's hardware using CAD and was responsible for its mechanical assembly. In particular, I focused on developing a robust drivetrain capable of navigating the challenging terrain of the field.



Shuichiro Mitsumoto

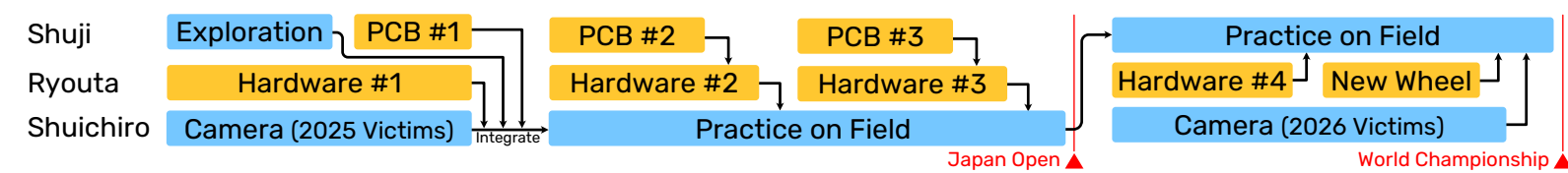
Software, Camera

I have been participating in RCJ since 2025. As Software Chief, I was responsible for software architecture and implementation. In particular, I focused on image processing to improve victim detection and support Cognitive Targets introduced this year.

Development Process

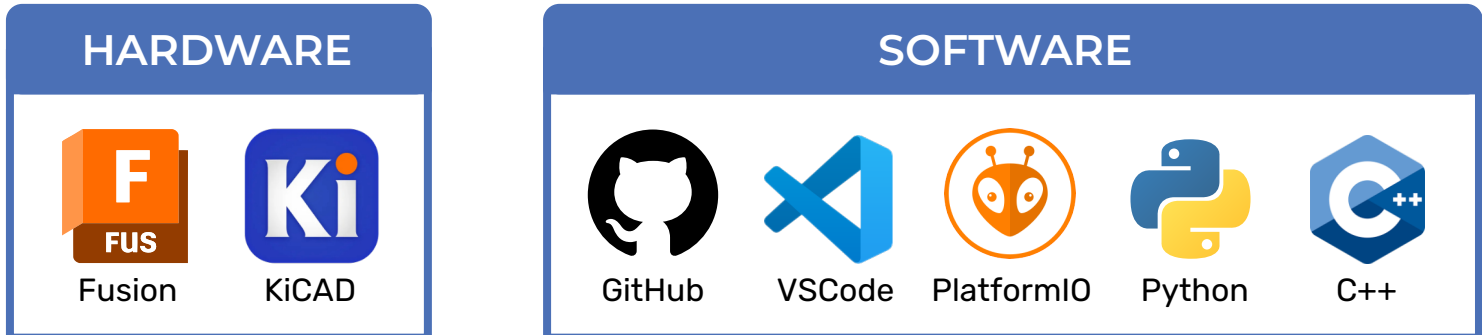
With less than one year to transition from Rescue Line to Rescue Maze, our primary challenge was to fully understand the competition requirements and develop a robot capable of meeting them all. To achieve this, we adopted an iterative development approach, rapidly identifying issues, testing potential solutions, and refining both the hardware and software through continuous experimentation and evaluation.

To accelerate development, we built two robots, allowing hardware and software development to **proceed in parallel**. Team members independently explored solutions to challenges, enabling fast iteration. Key software components, including victim detection and autonomous exploration algorithms, were also developed and tested on laptops before hardware completion, reducing later integration work.



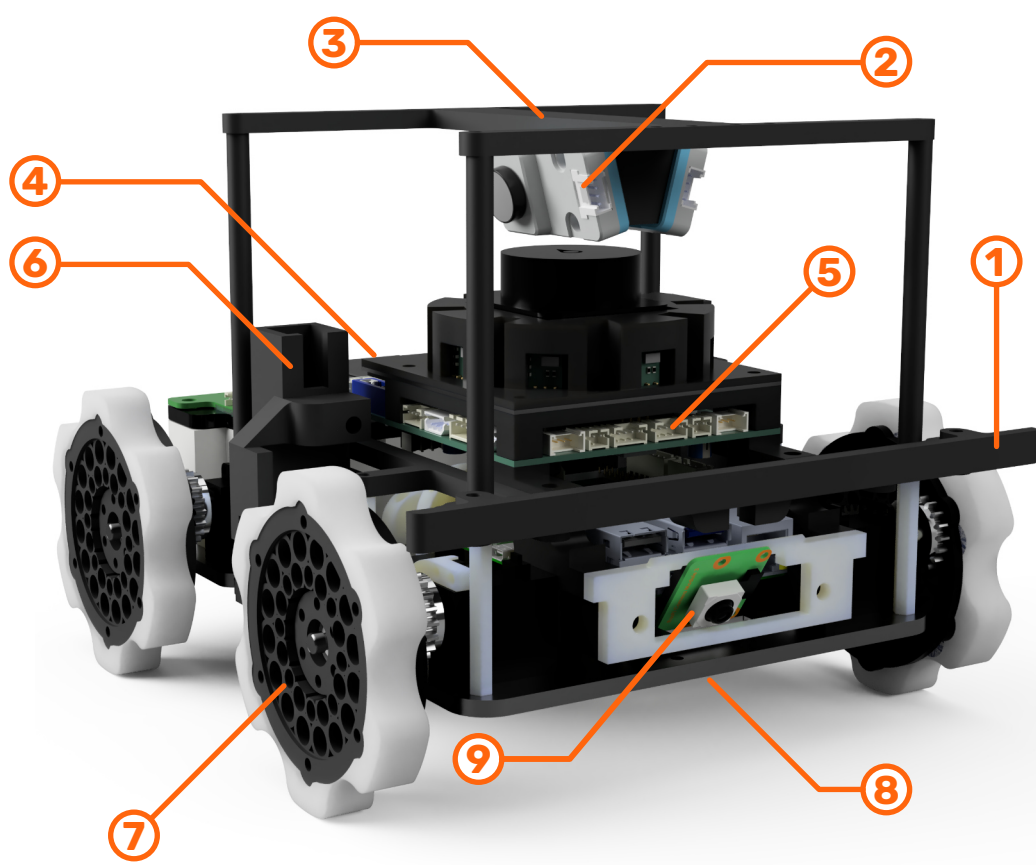
For hardware development, we used **Autodesk Fusion** for mechanical design and **KiCad** for PCB design. Sharing all design data within the team enabled efficient task distribution and collaboration.

For software development, we used **Visual Studio Code** and **GitHub**. The Raspberry Pi software and UnitV software were written in Python, while the STM32 firmware was developed in Platform IO using the Arduino framework. To maintain code quality and teamwork efficiency, we emphasized readability, documentation, and a well-structured codebase.



Throughout development, we documented issues and action items in our Engineering Journal, allowing us to clearly track our progress and maintain up-to-date understanding of the project's status. In addition, we incorporated the official RoboCupJunior Rescue CMS into our testing and practice sessions, which helped us analyze the robot's weaknesses and identify areas for improvement.

Hardware



① Bumper

Two tactile switches are mounted on the left and right sides of the robot. They are used to detect collisions with walls and obstacles.

② UnitV Camera x2

Two UnitV cameras are used for victim detection. LEDs provide consistent illumination independent of ambient lighting.

③ Top LED

A status indicator LED that flashes when a victim is detected and when the robot successfully returns to the start tile.

④ LCD Display

Displays the robot's current operating status and position within the generated map.

⑤ PCB Unit

The main electronics module containing the robot's primary sensors, processors, and communication interfaces.

A. Raspberry Pi 4 model B The main processor responsible for executing the robot's core software, including mapping, navigation, and decision-making.

B. T-mini Plus A A 360-degree LiDAR sensor used when wide-range environmental measurements are required, such as distinguishing walls from obstacles and generating local surroundings data.

C. VL53L4CX A single-point time-of-flight distance sensor used for high-speed and precise wall distance measurements.

D. BNO085 An IMU sensor used to estimate the robot's orientation and heading.

⑥ Wheel x4

The wheels are driven by STS3032 serial servo motors. Each wheel is a custom-designed deformable wheel featuring a silicone-molded tread, providing high traction and excellent terrain traversal capability.

⑦ Rescue Kit Releaser x2

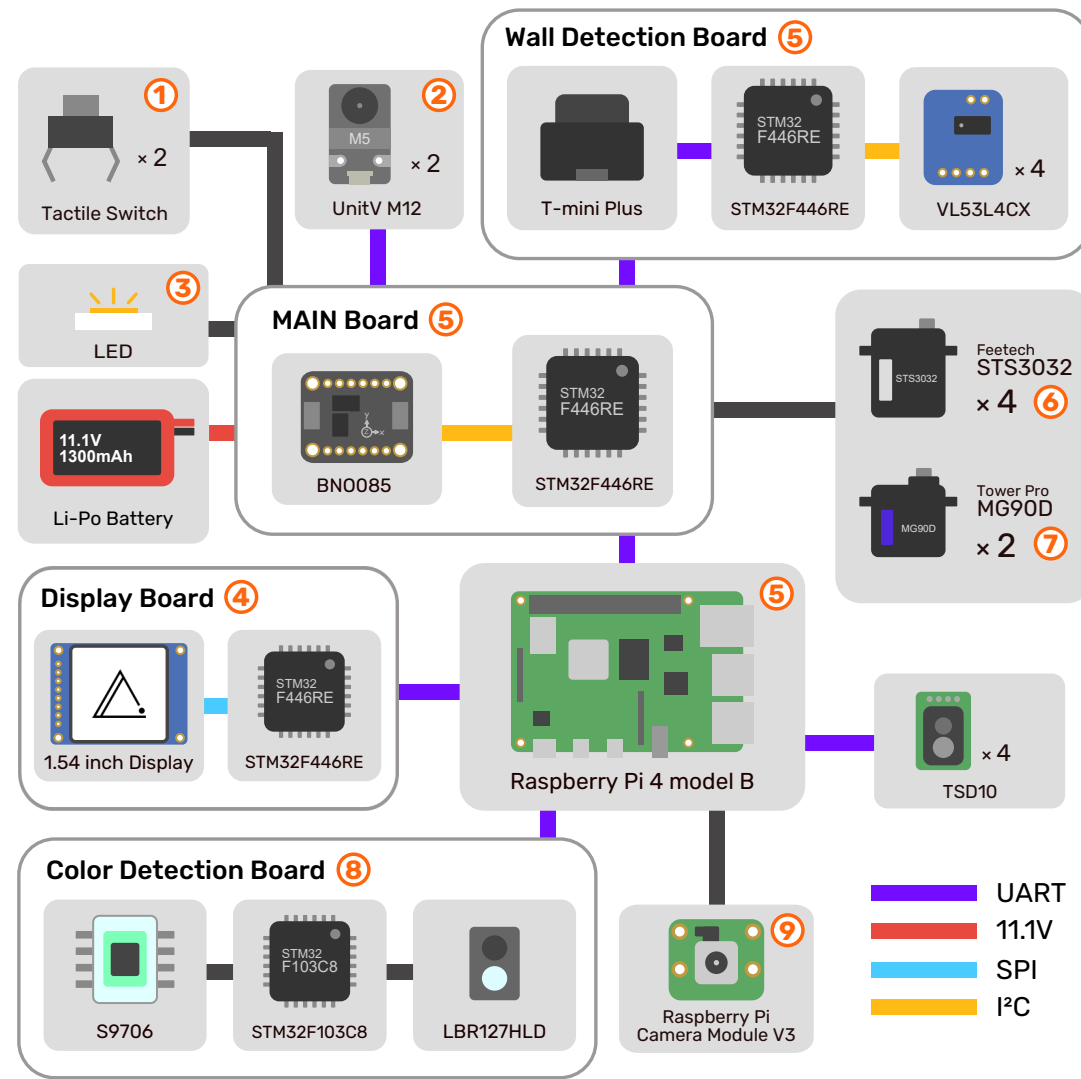
The rescue kit deployment mechanism. Four rescue kits are stored vertically on each side of the robot and are released individually using servo motors.

⑧ Color Detection Board

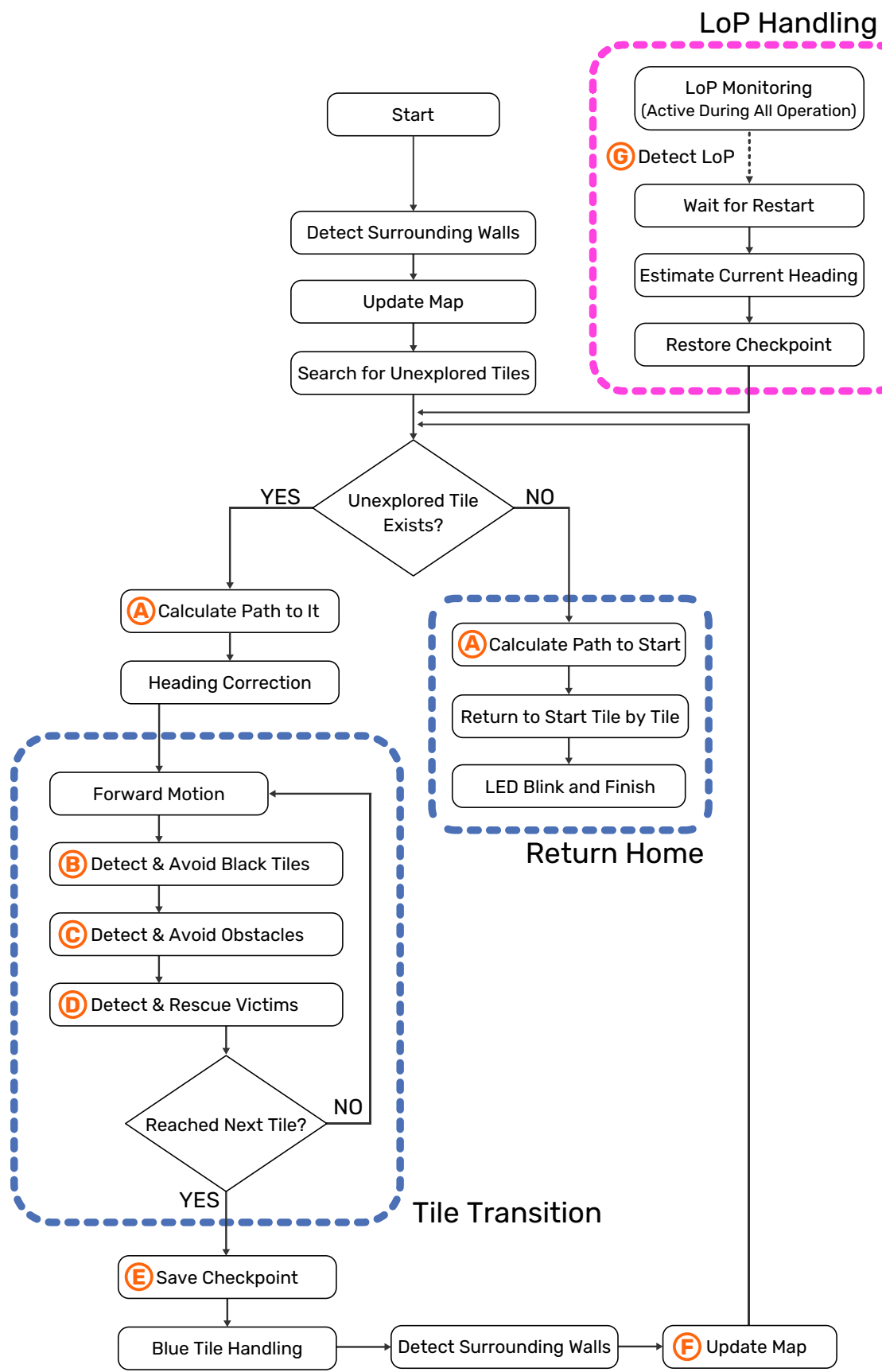
Used for tile classification. Visible-light sensors are used to identify black, blue, and red tiles, while infrared reflectance measurements are used to detect silver tiles.

⑨ Front Camera

An auxiliary camera used as a supplement to the Color Detection Board to assist in color tile identification.



Software



The main control logic is implemented in Python and executed on the Raspberry Pi.

The flowchart on the left shows the overall software flow.

The robot explores the maze one tile at a time while continuously updating its internal map based on detected walls. When a checkpoint is reached (⑤), the current map and exploration state are saved. In the event of a Lack of Progress (LoP, ⑥), the robot restores the most recent checkpoint data and resumes operation. During each tile transition, the robot continuously performs black tile detection (⑧), obstacle detection (③) and victim detection (②), executing the appropriate response whenever one of these elements is identified.

After all reachable tiles have been explored, the robot calculates the shortest path to the start tile and returns to complete the run.

The source code is available here:

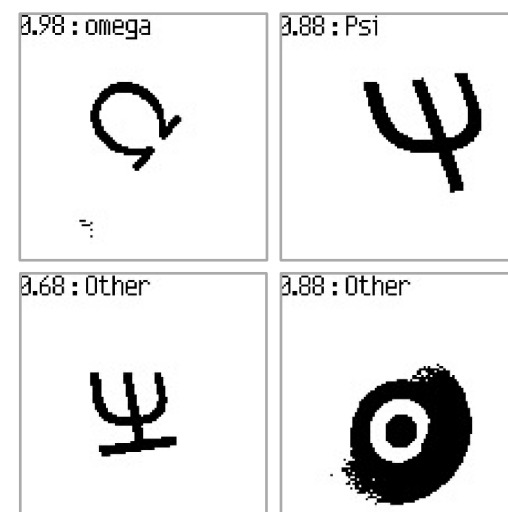


D Victim Detection

1. Letter victims

Captured images are binarized and converted to grayscale before being processed by a MobileNetV1 ($\alpha=0.75$) model. A victim is detected only when the model output exceeds a threshold across consecutive frames, reducing false detections.

To improve robustness, data augmentation techniques such as CutMix and image flipping were applied during training. In addition, Out-of-Distribution (OOD) detection using the Maximum Logit Score rejects low-confidence predictions, improving reliability in real-world environments. Experimental results showed that this threshold avoided approximately 40% (73/185) of false detections on fake victims.



Recognition Results

2. Cognitive Target

Candidates for cognitive targets are first detected from the image using a flood-fill method. Shape-based filtering is then applied, and the largest valid candidate is selected. After estimating the radius and center of the selected candidate, 3x3 pixels square regions are sampled in 12 directions, centered at points whose distances from the center are 0.1, 0.3, 0.5, 0.7, and 0.9 times the radius. The color of each layer and the total score are determined by voting based on these samples.

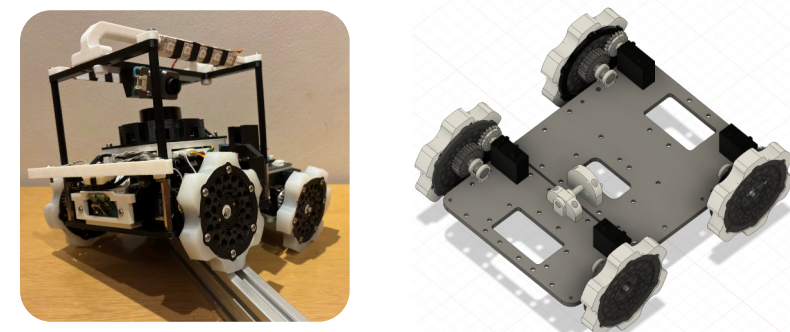
After that, the target is considered detected only when the agreement level of these votes exceeds a threshold and the same score is detected across consecutive frames.

Why we adopted a rule-based approach rather than CNN-based methods:

- Running multiple CNN models simultaneously is impractical due to memory and computational constraints.
- Each cognitive target label exhibits a wide variety of appearances, making robust classification difficult.
- A rule-based approach allows for easier tuning and adaptation compared to a CNN-based approach.

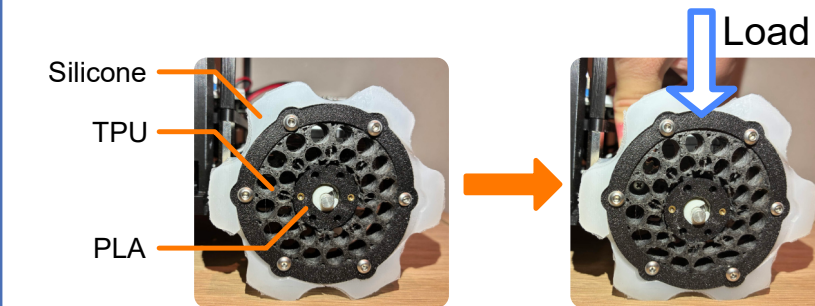
Twisting Mechanism

To overcome 2 cm bumps, we designed a compact twisting chassis that allows the front and rear sections of the robot to move independently. This keeps **all wheels in contact with the ground** and improves stability on uneven terrain. The mechanism also minimizes positional deviation while turning on bumps, enabling reliable navigation in challenging maze environments.



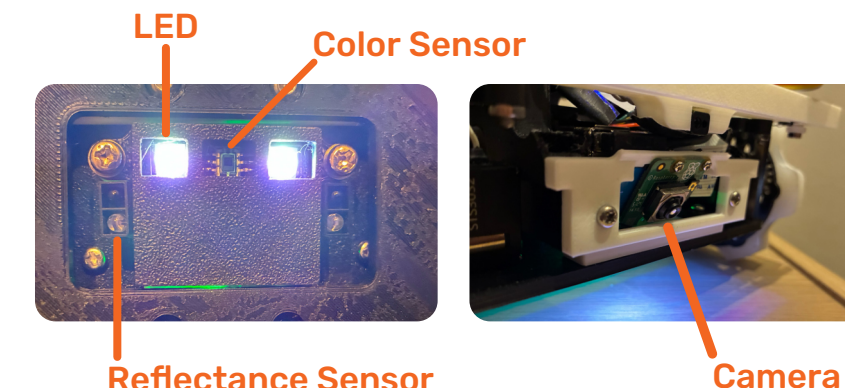
Deformable Wheels

To improve stability on bumps, ramps, and stairs, we developed a wheel (⑥) that provides suspension through deformation of the wheel itself. Unlike conventional spring-based suspension systems, this approach requires little additional space. The 80 mm wheel is constructed from TPU, PLA, and silicone. Its deformable structure absorbs shocks and maintains traction on uneven terrain, enabling reliable traversal of all obstacles encountered in the Rescue Maze field.



Floor Color Detection

The color sensor (⑧) may falsely detect black tiles when the robot is lifted by bumps, while the camera (⑨) alone cannot reliably distinguish black tiles from silver tiles. To address these limitations, we fused data from both sensors and classified a tile as black only when they agreed. This approach significantly reduced both false positives and missed detections.



Rescue Kit Deployment

The rescue kit is an 11 mm cubic capsule manufactured using a 3D printer. **A freely moving nut is enclosed inside the cube** to absorb impact energy when the kit contacts the floor or walls, reducing bounce and helping it stay near the target location.

The robot carries a total of eight rescue kits. Four kits are stacked vertically on each side of the robot and an MG90D servo motor pushes the kits out one at a time, enabling controlled and reliable deployment.



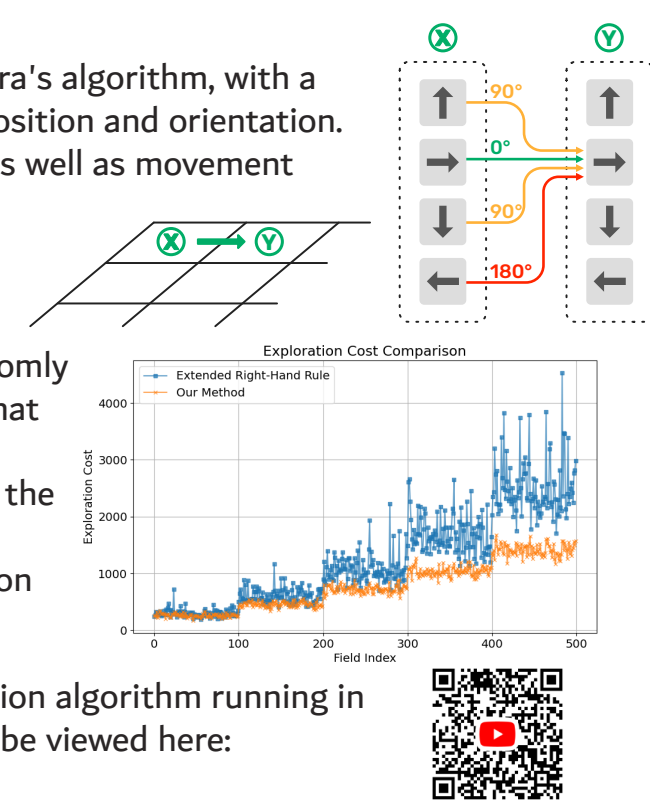
A Exploration Algorithm

The robot **repeatedly moves to the nearest unexplored tile** using the lowest-cost path and returns to the start tile after completing exploration.

Path planning is based on Dijkstra's algorithm, with a graph node representing both position and orientation. By incorporating turning costs as well as movement costs, the planner more accurately captures the robot's real behavior.

Simulation experiments on randomly generated maze fields showed that our method explores more efficiently and consistently than the Extended Right-Hand Rule, achieving lower overall exploration costs.

A demonstration of the exploration algorithm running in the simulation environment can be viewed here:



F 3D Mapping

To accurately map multi-level environments, we developed a **height-based layer matching system**. New layers are generated dynamically whenever the robot discovers a new floor through a ramp. Vertical displacement is estimated using gyro data and traveled distance, allowing layers at similar heights to be matched and treated as a single layer.

This approach enables reliable mapping even in complex fields with multiple interconnected ramps of **varying inclinations**. To prevent mapping errors caused by stairs, the system distinguishes stairs from actual floor transitions and avoids creating unnecessary layers when no layer change has occurred.

